

PROGRAMACIÓN I

Trabajo Práctico Integrador

Integrantes:

Brisa Chirino - Comisión 20 - Cohorte Marzo 2026

Clarisa Maria Boffelli - Comisión 16 - Cohorte Marzo 2026.

Entrega: 17 de junio de 2026

Índice

Introducción	4
Objetivos del trabajo	4
Consignas generales	4
Marco teórico	4
Listas	4
Diccionarios.....	5
Funciones.....	5
Condicionales	5
Estructuras repetitivas	5
Modularización.....	5
Validación de datos	5
Control de versiones.....	6
Gestión de proyectos	6
Ordenamientos.....	6
Estadísticas básicas.....	6
Archivos CSV.....	7
Herramientas empleadas.....	7
Python 3.x.....	7
Visual Studio Code.....	7
Git.....	7
GitHub Desktop y GitHub.....	7
Jira Software.....	8
Draw.io	8
Google Sheets y archivos CSV.....	8
README.md	8
Archivo .gitignore	8
Herramientas de Inteligencia Artificial	8
Google Docs	8
Decisiones técnicas y arquitectura.....	9
Planificación del proyecto	9
Diagramación inicial de los requisitos	9
Imagen 1: Diagrama de Gantt de la organización general del proyecto.	9
Imagen 2: Organigrama de los requisitos generales del programa.	10
Descripción del Proyecto	13
Arquitectura del programa.....	14
Estructura de almacenamiento de datos	14
Organización del repositorio	15
Estrategia de control de versiones	15
Consideraciones técnicas	15
Uso de listas paralelas	15

Algoritmo de ordenamiento.....	16
Validaciones	16
Diccionario de datos.....	16
Enlace al video explicativo.	20
Dificultades y conclusiones.....	20
Bibliografía	21

Introducción

El presente Trabajo Práctico Integrador fue desarrollado en el marco de la materia Programación I con el propósito de aplicar de forma práctica los conocimientos adquiridos durante la cursada. A través de la implementación de un programa en Python, se trabajó con estructuras de datos, funciones, validaciones, archivos CSV y técnicas de procesamiento de información para gestionar datos de distintos países.

Además del desarrollo del programa, el trabajo permitió poner en práctica herramientas y metodologías utilizadas durante el proceso de desarrollo de software, tales como la planificación de tareas, la elaboración de diagramas, la documentación técnica, el control de versiones mediante Git y GitHub y la organización del trabajo colaborativo mediante Jira Software.

Objetivos del trabajo

Desarrollar un programa en Python que permita gestionar información sobre países, aplicando listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema debe ser capaz de leer datos desde un archivo CSV, realizar consultas y generar indicadores clave a partir del dataset. El objetivo principal es afianzar el uso de estructuras de datos, modularización con funciones y técnicas de filtrado/ordenamiento, aplicando los conceptos aprendidos en Programación 1.

Consignas generales

- Lenguaje: Python 3.x
- Estructuras: listas, diccionarios, funciones.
- Archivos: lectura desde CSV.
- Código claro, comentado y modularizado (una función = una responsabilidad).
- Validaciones de entradas y manejo básico de errores.
- Trabajo en equipos de 2 personas.

Marco teórico

Listas

Una lista es una estructura de datos que permite almacenar una colección ordenada de elementos. Estos elementos pueden ser de distintos tipos y se accede a ellos mediante índices. Las listas permiten agregar, modificar, eliminar y recorrer datos de forma sencilla.

En este trabajo se utilizaron listas paralelas para almacenar la información de cada país: nombre, población, superficie y continente. Cada posición de las listas representa un mismo país, permitiendo mantener los datos relacionados entre sí.

Diccionarios

Un diccionario es una estructura de datos que permite almacenar información mediante un sistema de pares clave-valor. Cada clave es única y permite acceder de forma rápida al valor asociado. En este trabajo se empleó para contabilizar la cantidad de países pertenecientes a cada continente utilizando el nombre del continente como clave y la cantidad de países como valor.

Funciones

Una función es un bloque de código reutilizable diseñado para realizar una tarea específica. Su utilización permite organizar mejor el programa, reducir la repetición de instrucciones y facilitar el mantenimiento del código. En este trabajo se desarrollaron funciones para la carga de datos desde archivos CSV, validación de entradas, búsqueda de países, filtrado de información, ordenamientos, cálculos estadísticos y gestión de los distintos menús del sistema.

Condicionales

En programación, una estructura condicional permite ejecutar diferentes bloques de código según se cumpla o no una determinada condición. En Python se implementan principalmente mediante las sentencias `if`, `elif`, y `else`.

Su utilización fue fundamental para validar datos ingresados por el usuario, controlar errores, gestionar opciones de menú y determinar distintas acciones dentro del programa.

Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar un bloque de instrucciones varias veces mientras se cumpla una condición o para recorrer una colección de datos. En Python se utilizan principalmente los bucles `for` y `while`. En este proyecto se emplearon para recorrer listas, buscar países, aplicar filtros, realizar ordenamientos y mantener el funcionamiento continuo de los menús interactivos.

Modularización

La modularización consiste en dividir un programa en componentes o funciones independientes, cada una con una responsabilidad específica. Esta práctica mejora la legibilidad del código, facilita su mantenimiento y favorece la reutilización de funcionalidades.

El sistema fue desarrollado siguiendo este criterio, organizando cada responsabilidad del programa en funciones independientes.

Validación de datos

La validación de datos es el proceso mediante el cual se verifica que la información ingresada sea correcta antes de ser utilizada por el programa. Esto ayuda a prevenir errores y garantiza la consistencia de los datos almacenados.

En este trabajo se implementaron validaciones para nombres de países, población, superficie, continentes y opciones de menú, además de controles para evitar datos vacíos o valores inválidos.

Control de versiones

El control de versiones es una práctica que permite registrar y gestionar los cambios realizados sobre un proyecto. Facilita el trabajo colaborativo y permite recuperar versiones anteriores cuando es necesario. Para ello se utilizó Git como sistema de control de versiones y GitHub como plataforma para almacenar el repositorio y coordinar el trabajo.

Gestión de proyectos

La gestión de proyectos permite planificar, organizar y supervisar las tareas necesarias para alcanzar los objetivos planteados. En el desarrollo del trabajo se utilizó Jira Software para registrar tareas, organizar actividades, realizar el seguimiento del avance, asignar responsabilidades y monitorear el progreso del proyecto.

Ordenamientos

Los algoritmos de ordenamiento permiten reorganizar los datos según determinados criterios, facilitando su consulta y análisis.

En este proyecto se implementaron los siguientes ordenamientos de países, ofreciendo la posibilidad de visualizar los resultados en forma ascendente o descendente.

- Ordenamiento de países por nombre, con la opción de hacerlo en orden alfabético ascendente o descendente.
- Ordenamiento de países por población, con la opción de hacerlo en orden numérico ascendente o descendente.
- Ordenamiento de países por superficie, tanto en orden numérico ascendente o descendente.

Estadísticas básicas

Las estadísticas básicas permiten obtener información relevante a partir de un conjunto de datos mediante operaciones matemáticas sencillas.

A partir de los datos almacenados se obtuvieron los siguientes indicadores:

- País con mayor y menor población.
- Promedio de población entre el total de países del catálogo.
- Promedio de superficie.
- Cantidad de países registrados por continente.

Archivos CSV

Un archivo CSV es un archivo de texto que guarda datos en forma de tabla, donde cada fila representa un registro y cada columna se separa con comas u otro delimitador. Suele usarse para exportar e importar datos entre aplicaciones, asegurando que los datos persistan en el programa. En este proyecto se utilizó el archivo datos.csv como fuente inicial de información. Los registros corresponden a cinco países cuyos datos fueron obtenidos de la base de datos de las Naciones Unidas.

Herramientas empleadas

Python 3.x

Python fue el lenguaje de programación utilizado para el desarrollo del programa. Se eligió por su sintaxis clara, facilidad de aprendizaje y amplia disponibilidad de bibliotecas para el procesamiento de datos y manejo de archivos.

Visual Studio Code

Visual Studio Code fue el entorno de desarrollo utilizado para la escritura, edición y depuración del código fuente. Este editor permite trabajar con múltiples lenguajes de programación, incorporar extensiones y organizar proyectos de manera eficiente.

Git

Git se utilizó como sistema de control de versiones para registrar y administrar la evolución del proyecto a lo largo de su desarrollo. Su implementación permitió mantener un historial de modificaciones, trabajar mediante ramas independientes y facilitar la integración de nuevas funcionalidades de forma organizada.

GitHub Desktop y GitHub

GitHub y GitHub Desktop fueron empleados como herramientas de apoyo para el trabajo colaborativo y la gestión del repositorio. GitHub permite a los desarrolladores almacenar código en la nube de forma segura, compartir los avances realizados y mantener un registro de los cambios efectuados por cada integrante. Por su parte, GitHub Desktop facilitó la gestión de las tareas relacionadas con Git mediante una interfaz gráfica, simplificando operaciones como commits, sincronización de cambios y administración de ramas.

Durante el desarrollo del trabajo se aplicaron prácticas de control de versiones tales como:

- Creación y utilización de ramas de trabajo.
- Realización de commits descriptivos para registrar avances.
- Sincronización de cambios entre el repositorio local y el repositorio remoto.
- Integración de modificaciones mediante Git.

Jira Software

Jira Software fue utilizado para la planificación, organización y seguimiento de las tareas del proyecto. A través de esta herramienta se registraron actividades, se asignaron responsabilidades y se monitoreo el avance de las distintas etapas del desarrollo.

Draw.io

Draw.io se utilizó para la elaboración de diagramas y representaciones gráficas relacionadas con la planificación y estructura del proyecto. Esta herramienta permitió visualizar de forma clara los requisitos, la organización de tareas y distintos aspectos del diseño del sistema.

Google Sheets y archivos CSV

Google Sheets se utilizó para organizar y estructurar inicialmente los datos de los países. Posteriormente, esta información fue exportada al archivo [datos.csv](#), utilizado por el programa como fuente de datos para la carga inicial de información.

README.md

Se elaboró un archivo [README.md](#) con el objetivo de documentar el proyecto. Este documento incluye la descripción general del sistema, instrucciones de ejecución, funcionalidades implementadas, estructura del repositorio y aspectos técnicos relevantes.

Archivo .gitignore

Se configuró un archivo [.gitignore](#) para excluir del repositorio archivos temporales, configuraciones del entorno de desarrollo y otros elementos que no forman parte del código fuente del proyecto. Esto contribuyó a mantener el repositorio más organizado y limpio.

Herramientas de Inteligencia Artificial

Como apoyo durante el desarrollo se utilizaron herramientas de inteligencia artificial para consultas técnicas, resolución de dudas, revisión de código, búsqueda de alternativas de implementación y asistencia en la elaboración de documentación.

Las herramientas empleadas fueron:

- ChatGPT
- Claude
- Google Gemini
- DeepSeek

Google Docs

Google Docs (Documentos de Google) fue utilizado para la elaboración colaborativa de la documentación del proyecto. Esta herramienta permite redactar, editar y compartir el informe de manera simultánea, facilitando la organización del contenido, el seguimiento de modificaciones y el trabajo conjunto entre las integrantes del equipo.

Decisiones técnicas y arquitectura

Planificación del proyecto

Antes de comenzar la implementación del programa se realizó un análisis de los requerimientos establecidos en la consigna. A partir de ellos se identificaron las funcionalidades principales que debía ofrecer el sistema, tales como la gestión de países, las búsquedas, los filtros, los ordenamientos y la generación de estadísticas.

Posteriormente se organizaron las tareas de desarrollo utilizando Jira Software, lo que permitió planificar las actividades, distribuir responsabilidades y realizar un seguimiento del avance del proyecto.

Diagramación inicial de los requisitos

Con el objetivo de comprender la estructura general del sistema y planificar su desarrollo, se elaboraron diagramas mediante Draw.io. Estos diagramas permiten representar visualmente las funcionalidades requeridas, la organización de los menús y la relación entre los distintos componentes del programa.

La utilización de diagramas facilitó la definición de la lógica de funcionamiento antes de iniciar la etapa de codificación.

Planificación del desarrollo del programa

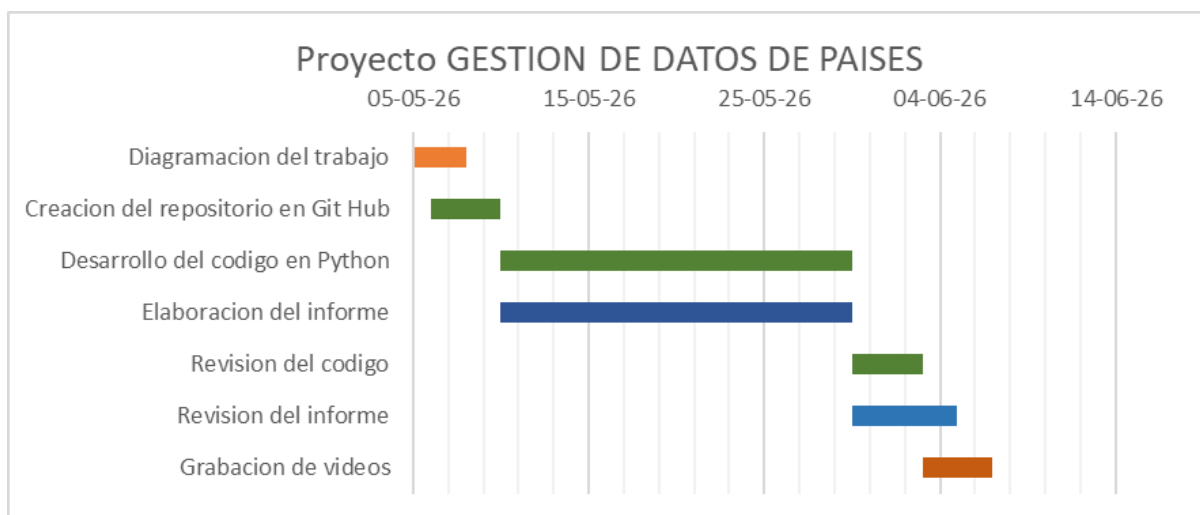


Imagen 1: Diagrama de Gantt de la organización general del proyecto.

Diagramación inicial de los requisitos

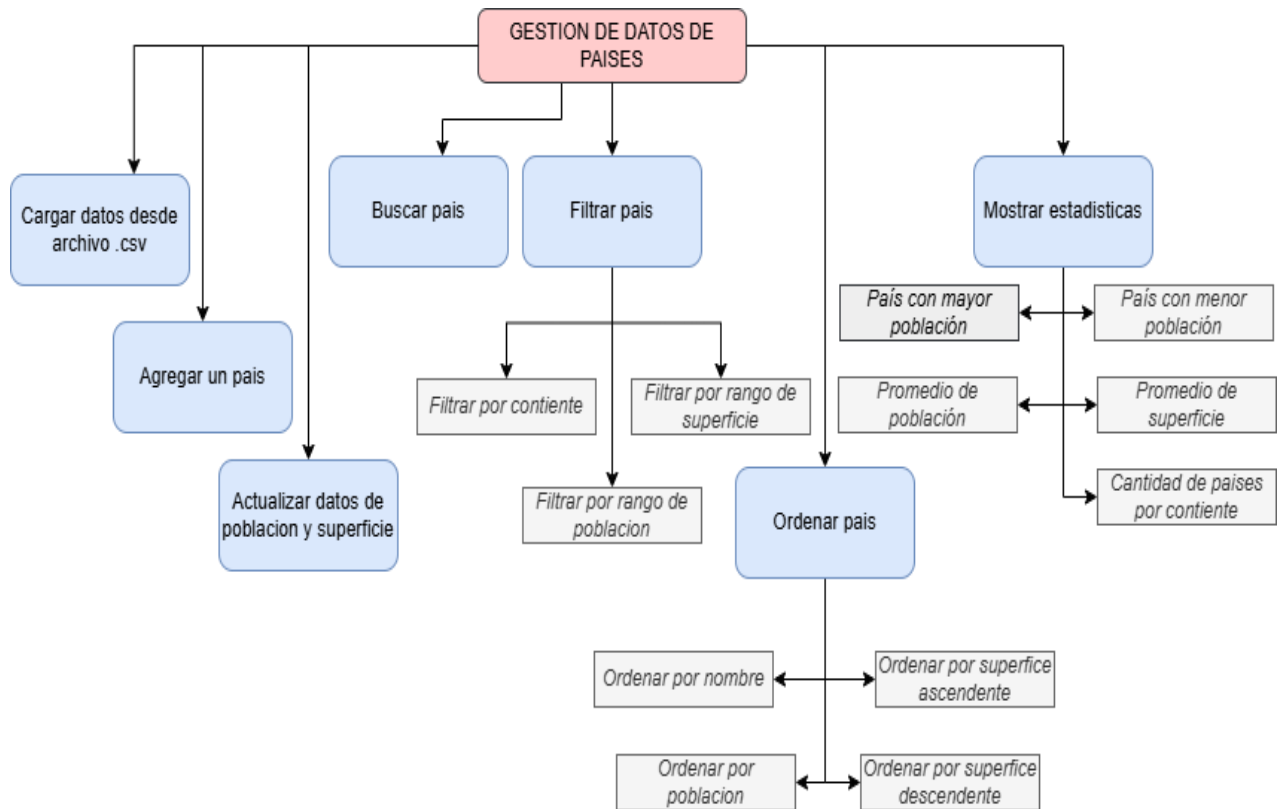


Imagen 2: Organigrama de los requisitos generales del programa.

Organización del desarrollo en Jira Software.

Con el objetivo de planificar y organizar las actividades del proyecto, se utilizó Jira Software como herramienta de gestión de tareas. A través de esta plataforma se registraron las distintas actividades necesarias para el desarrollo del trabajo, permitiendo organizar las tareas pendientes, realizar el seguimiento de su avance y visualizar el estado general del proyecto.

Clave de elemento de actividad	Tipo de actividad	Prioridad de la actividad	Resumen
GDDP-1	Story	Medium	Como usuario quiero que haya un catalogo de datos de paises para consultar.
GDDP-10	Subtarea	Medium	Ordenar países por nombre.
GDDP-11	Subtarea	Medium	Ordenar países por población.
GDDP-12	Subtarea	Medium	Ordenar países por superficie en orden ascendente.
GDDP-13	Subtarea	Medium	Ordenar países por superficie en orden descendente.
GDDP-14	Historia	Medium	Como usuario quiero visualizar estadísticas generadas a partir de los datos del catálogo de países.
GDDP-15	Subtarea	Medium	Mostrar país con mayor población.
GDDP-16	Subtarea	Medium	Mostrar país con menor población.
GDDP-17	Subtarea	Medium	Mostrar el promedio de superficie de todos los países del catálogo.
GDDP-18	Subtarea	Medium	Mostrar el promedio de población de todos los países del catálogo.
GDDP-19	Subtarea	Medium	Mostrar la cantidad de países por continente.
GDDP-2	Story	Medium	Como usuario quiero poder agregar un país al catalogo para ampliarlo.
GDDP-20	Epic	Medium	Cargar datos desde archivo .csv
GDDP-21	Epic	Medium	Agregar un país
GDDP-22	Epic	Medium	Actualizar datos de población y superficie.
GDDP-23	Epic	Medium	Buscar países
GDDP-24	Epic	Medium	Filtrar país
GDDP-25	Epic	Medium	Ordenar país
GDDP-26	Epic	Medium	Mostrar estadísticas
GDDP-27	Epic	Medium	Revision final y aprobacion
GDDP-4	Story	Medium	Como usuario quiero busca un país en el catálogo para ver la información de dicho país.
GDDP-5	Historia	Medium	Como usuario quiero filtrar los países del catálogo por diferentes criterios para visualizar la información.
GDDP-6	Subtarea	Medium	Filtrar los países por continente.
GDDP-7	Subtarea	Medium	Filtrar los países por rango de población.
GDDP-8	Subtarea	Medium	Filtrar los países por rango de superficie.
GDDP-9	Story	Medium	Como usuario quiere ordenar los países del catálogo por diferentes criterios para visualizar la información.

Imagen 3: Historias de usuario, subtareas, tareas, y épicas del proyecto en Jira Software.

Durante el desarrollo del proyecto, las tareas registradas fueron actualizadas según su estado de ejecución. Las actividades completadas se marcaron como finalizadas, permitiendo mantener un

seguimiento ordenado del trabajo realizado y visualizar el avance alcanzado en cada etapa del proyecto.

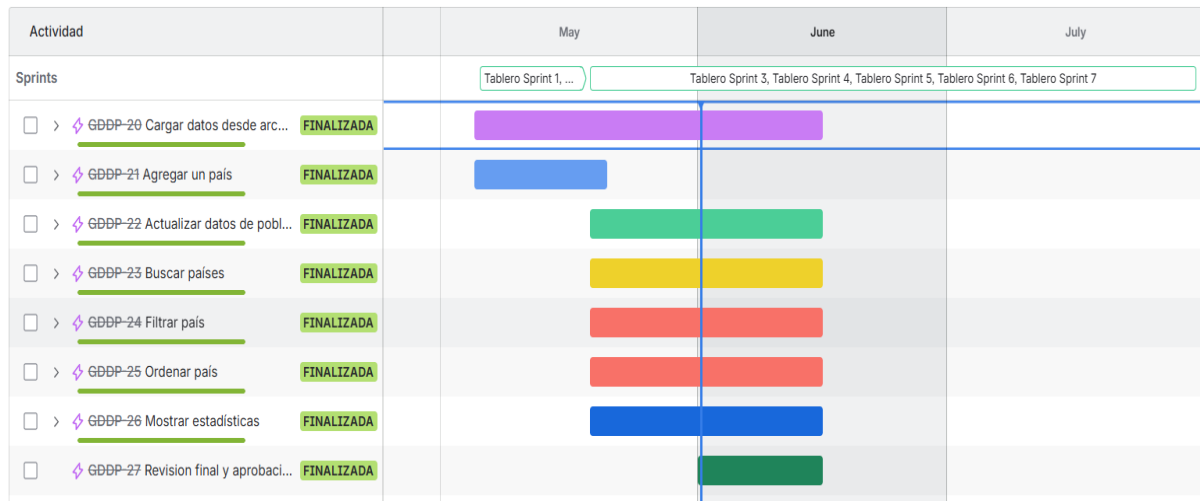


Imagen 4: Cronograma de sprints en Jira Software.

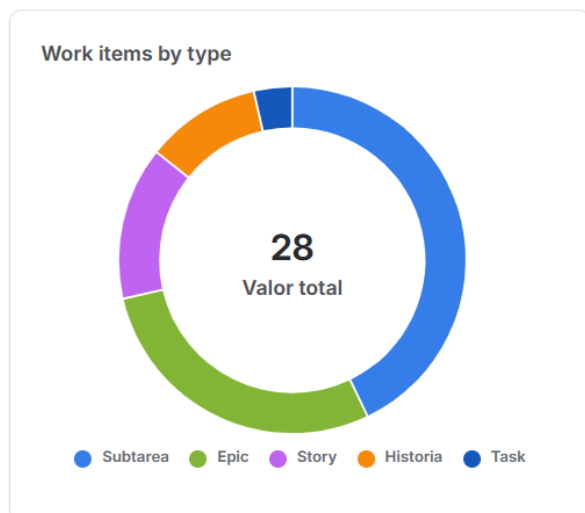


Imagen 5: Distribución de los elementos por tipo.

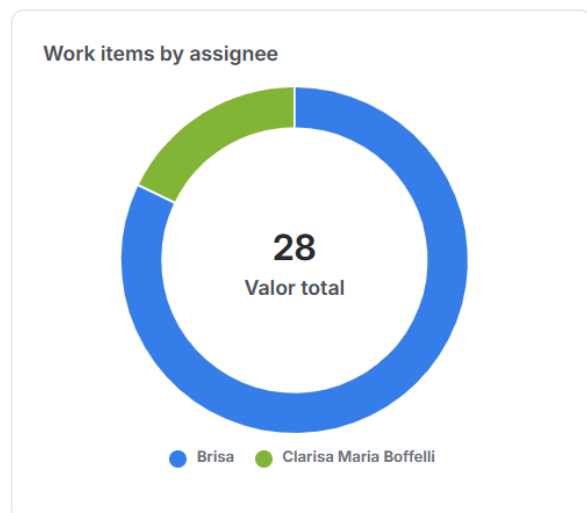


Imagen 6: Distribución de las tareas por colaborador

Descripción del Proyecto

El programa desarrollado permite gestionar información de distintos países a partir de los datos almacenados en un archivo CSV. Mediante una interfaz de consola, el usuario puede realizar operaciones de consulta, búsqueda, actualización, filtrado, ordenamiento y obtención de estadísticas sobre los países registrados.

Con el objetivo de mejorar la organización y facilitar la navegación, las funcionalidades fueron agrupadas en una estructura jerárquica compuesta por un menú principal y diferentes submenús temáticos. Esta organización permite acceder de manera clara y ordenada a cada una de las opciones disponibles, favoreciendo la usabilidad y el mantenimiento del sistema.

La siguiente figura muestra la estructura general de menús implementada y la distribución de las funcionalidades dentro del programa.

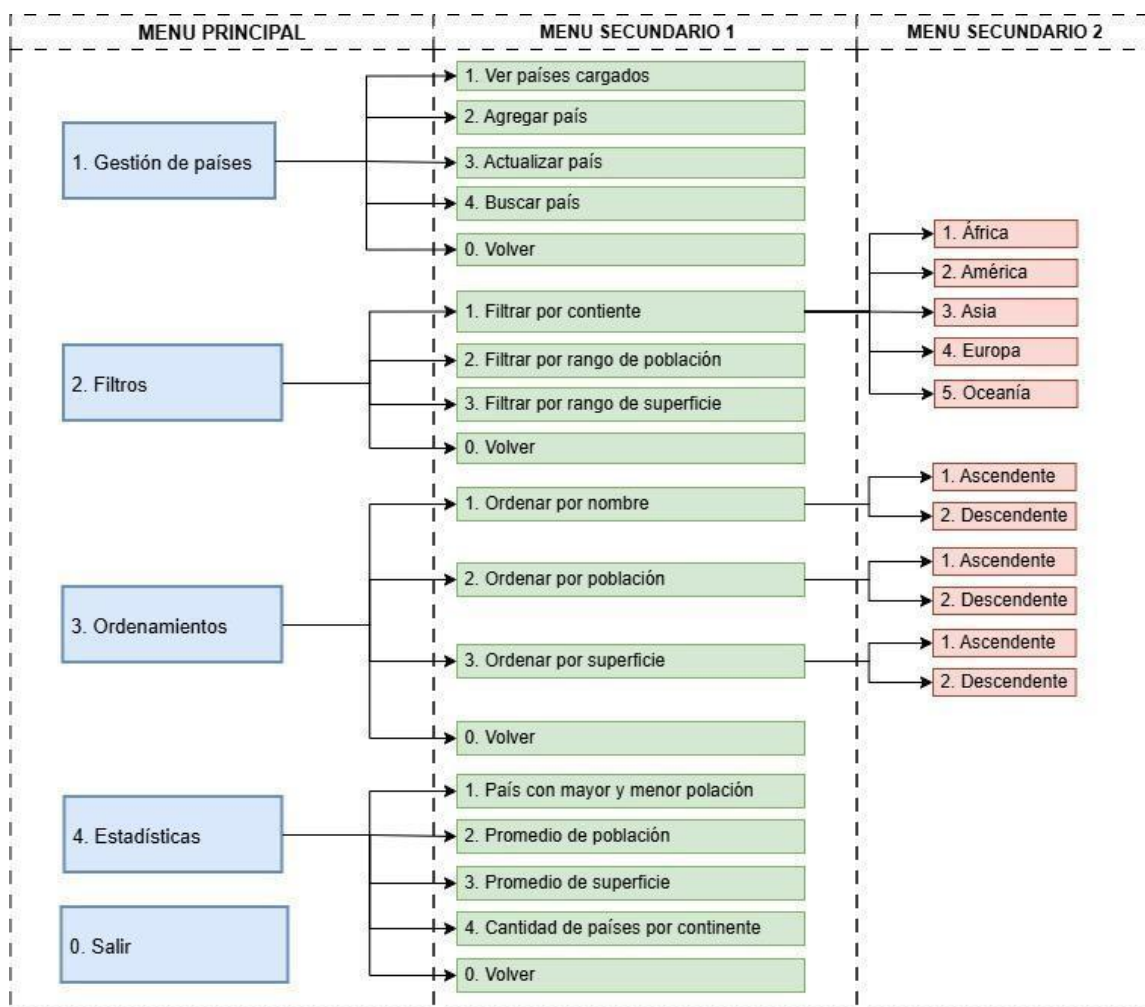


Imagen 7: Diagramación del menú.

Arquitectura del programa

El sistema fue desarrollado siguiendo una estructura modular basada en funciones independientes. Cada función fue diseñada para cumplir una única responsabilidad específica, favoreciendo la organización del código, su mantenimiento y su reutilización.

La arquitectura general del programa se compone de los siguientes módulos funcionales:

- Carga de datos desde archivo CSV.
- Gestión de países (alta, actualización y búsqueda).
- Filtros de información.
- Ordenamientos.
- Estadísticas.
- Menús de navegación y control del programa.

Esta organización permitió separar claramente las distintas responsabilidades del sistema y simplificar su desarrollo.

Estructura de almacenamiento de datos

Para almacenar la información de los países se utilizaron listas paralelas sincronizadas. Cada posición de las listas representa un mismo país, manteniendo asociada la información correspondiente a:

- Nombre.
- Población.
- Superficie.
- Continente.

Esta decisión fue tomada debido a que el trabajo requería la utilización de listas y permitía aplicar de forma práctica conceptos de búsqueda, filtrado y ordenamiento vistos durante la cursada.

Además, para el cálculo de la cantidad de países por continente se utilizó un diccionario, permitiendo asociar cada continente con su correspondiente contador.

Organización del repositorio

El proyecto fue organizado mediante un repositorio GitHub que contiene los archivos necesarios para su ejecución y documentación.

La estructura principal del repositorio está compuesta por:

TPI_PAISES/

```
|  
|— datos.csv  
|— gestion_paises.py  
|— README.md  
|— informe.pdf  
|— .gitignore
```

Esta organización permitió mantener separados el código fuente, los datos utilizados por el programa y la documentación del proyecto.

Estrategia de control de versiones

Durante el desarrollo se utilizó Git como sistema de control de versiones y GitHub como repositorio remoto.

Para evitar conflictos y mantener una mejor organización del trabajo, se implementó una estrategia basada en ramas de desarrollo individuales. Cada integrante realizó modificaciones sobre su propia rama y posteriormente los cambios fueron integrados al proyecto principal mediante las herramientas de Git y GitHub.

Asimismo, se utilizaron commits descriptivos para registrar los avances realizados en cada etapa del desarrollo.

Consideraciones técnicas

Uso de listas paralelas

Se utilizaron listas paralelas para almacenar la información de los países.

Esto permitió:

- Mantener sincronizados todos los datos.
- Facilitar los ordenamientos.
- Simplificar los recorridos por índice.
- Organizar mejor la lógica del programa.

Algoritmo de ordenamiento

Se implementó un algoritmo de ordenamiento manual mediante intercambio de posiciones entre listas sincronizadas.

Validaciones

Se desarrollaron funciones específicas para validar:

- nombres,
- población,
- superficie,
- continentes.

Además, se incorporó manejo de errores para evitar fallos durante la ejecución del sistema.

Diccionario de datos

Con el objetivo de documentar los componentes utilizados en el desarrollo del programa, se elaboró el siguiente diccionario de datos. En él se detallan las estructuras de almacenamiento, variables y funciones implementadas, indicando para cada elemento su nombre, tipo, descripción, finalidad dentro del sistema y las validaciones aplicadas cuando corresponda.

Nombre	Tipo	Dato	Descripción	Validación
nombres	Lista	str	Almacena nombres de países.	Caracteres alfabéticos con espacios. Longitud de 2 a 50 caracteres.
poblaciones	Lista	int	Almacena población de cada país	Números enteros mayores a 0.
continentes	Lista	str	Almacena continente a que pertenece el país.	Caracteres alfabéticos. ["África", "América", "Asia", "Europa", "Oceanía"]
superficies	Lista	int	Almacena superficie en km2 del país.	Caracteres numéricos.
continentes_validos	Lista	str	Almacena los nombres de los continentes.	["África", "América", "Asia", "Europa", "Oceanía"]

Tabla 1: Diccionario de datos (listas)..

Nombre	Tipo	Dato	Descripción	Validación
nombre	Variable	str	Nombre del país ingresado por el usuario al catálogo.	Usa la función <validar_nombre>
numero	Variable	int	Población del país ingresado por el usuario.	Usa la función <validar_poblacion>
superficie	Variable	int	Superficie del país ingresado por el usuario.	Usa la función <validar_superficie>
opcion	Variable	str	Variable de control utilizada en todos los menús para capturar la entrada del usuario.	
nombre_buscar	Variable	str	País que el usuario quiere localizar para modificar sus datos.	Usa la función <buscar_por_nombre>
minimo	Variable	int	Valor de rango inferior en las funciones de filtrado de superficie y de población.	Usa la función <validar_superficie> y <validar_poblacion> respectivamente.
maximo	Variable	int	Valor de rango superior en las funciones de filtrado de superficie y de población.	Usa la función <validar_superficie> y <validar_poblacion> respectivamente.
mayor	Variable	int	Almacena el valor mayor de población.	
menor	Variable	int	Almacena el valor menor de población.	
promedio	Variable	float	Resultado del cálculo del valor promedio de la población y superficie de todos los países en el catálogo	Hasta 2 cifras decimales. Indicar en “km²” en caso de superficie, “habitantes” en caso de población.

Tabla 2: Diccionario de datos (variables).

Nombre	Tipo	Descripción	Validación
contador	Diccionario	Almacena como clave el continente y como valor el número de países pertenecientes a ese continente.	

Tabla 3: Diccionario de datos (diccionarios).

Nombre	Tipo	Descripción	Validación
cargar_csv	Funcion	Carga los datos del archivo datos.csv	Encoding utf-8. Muestra mensaje de carga correcta o de error usando estructura try/except.
mostrar_pais	Funcion	Muestra los datos de un país en el orden “Nombre:”, “Población:”, Superficie:” y “Continente:”	
validar_orden	Función	Seleccionar el orden ascendente o descendente en los submenús de Ordenamientos.	[“1”, “2”]. Muestra mensaje de error si la opción no es válida.
validar_nombre	Función	Valida el ingreso del usuario del nombre del país en la variable <nombre>	Usa .strip() y .title() para normalizar. No puede estar vacío. Entre 2 y 50 caracteres alfabéticos. Muestra mensajes de error para cada desvío.
validar_poblacion	Funcion	Valida el ingreso del usuario de la población del país.	Usa .strip() y .lower() para normalizar. Permite que se ingresen números acompañados de texto. Identifica el texto y genera el número entero positivo correspondiente. Muestra mensajes de error.
validar_superficie	Funcion	Valida el ingreso del usuario de la superficie del país.	Usa .strip() para normalizar. Permite ingreso de datos con puntos y comas y los normaliza. Devuelve el número entero positivo. Muestra mensajes de error.
validar_continente	Funcion	Muestra al usuario las opciones válidas de continentes al que pertenece el país usando la lista <continentes_validos>	Permite seleccionar entre 1. África, 2. América, 3.Asia, 4.Europa y 5.Oceanía. [“1”, “2”, “3”, “4”, “5”]. Ingreso numérico. Muestra mensaje de error.
mostrar_todos_los_paises	Funcion	Muestra el listado de los países en el catálogo usando la lista <nombres>.	Valida que haya países cargados. Muestra mensaje si no hay países en el catálogo.
agregar_pais	Funcion	Permite al usuario agregar los datos de un país nuevo en el catálogo a través de las listas <nombres>, <poblaciones>, <superficies> y <continentes>	Verifica que el país agregado no exista en la lista <nombres>. Valida cada ingreso con la función correspondiente.

actualizar_pais	Funcion	Permite al usuario buscar un país y modificar los datos de población y superficie.	Verifica que la lista <nombres> no está vacía. Valida los ingresos con las funciones <validar_poblacion> y <validar_continente>. Muestra error si no encuentra el país.
buscar_por_nombre	Funcion	Busca el nombre de un país en el listado <nombres>.	Verifica que la lista <nombres> no está vacía. Muestra mensajes de error.
filtrar_por_continente	Funcion	Muestra los países que corresponden a cada continente según la elección del usuario.	Valida el continente con la función <validar_continente>. Muestra mensaje si no se encuentran países en el continente seleccionado.
filtrar_por_rango_poblacion	Funcion	Muestra los listados de países que presentan una población entre el rango dado por las variables <minimo> y <maximo> determinadas por el usuario.	Valida las variables con las funciones <validar_poblacion>. Valida que la variable <maximo> sea mayor a <minimo>. Muestra mensajes de error.
filtrar_por_rango_superficie	Funcion	Muestra los listados de países que presentan una superficie entre el rango dado por las variables <minimo> y <maximo> definidas por el usuario.	Valida las variables con las funciones <validar_superficie>. Valida que la variable <maximo> sea mayor a <minimo>. Muestra mensajes de error.
ordenar_por_nombre	Funcion	Ordena los nombres de los países por criterios ascendente o descendente determinado por el usuario.	Valida el ingreso con la función <validar_orden>. Muestra mensaje de confirmación.
ordenar_por_poblacion	Funcion	Ordena los países por población siguiendo un criterio ascendente o descendente determinado por el usuario.	Valida el ingreso con la función <validar_orden>. Muestra mensaje de confirmación.
ordenar_por_superficie	Funcion	Ordena los países por superficie siguiendo un criterio ascendente o descendente determinado por el usuario.	Valida el ingreso con la función <validar_orden>. Muestra mensaje de confirmación.
pais_mayor_menor_poblacion	Funcion	Muestra los países que cumplen los criterios de mayor y menor población en el catálogo. Emplea funciones max() y min() en la lista <poblaciones>	Valida que existan países en el listado <nombres>.

promedio_poblacion	Funcion	Calcula el promedio del total de datos de cantidad de habitantes en el listado <poblaciones>.	Valida que existan países en el listado <nombres>. Muestra resultados con 2 decimales.
promedio_superficie	Funcion	Calcula el promedio del total de datos de superficie en el listado <superficies>	Valida que existan países en el listado <nombres>. Muestra resultados con 2 decimales.
cantidad_paises_por_continente	Funcion	Muestra el total de países por continente empleando el diccionario <contador>	
menu_gestion	Funcion	Define la estructura del submenú: “1. Gestion de países”	[“0”, “1”, “2”, “3”, “4”]. Muestra mensaje de error.
menu_filtros	Funcion	Define la estructura del submenú: “2. Filtros”	[“0”, “1”, “2”, “3”]. Muestra mensaje de error.
menu_ordenamientos	Funcion	Define la estructura del submenú: “3. Ordenamientos”	[“0”, “1”, “2”, “3”]. Muestra mensaje de error.
menu_estadisticas	Funcion	Define la estructura del submenú: “4. Estadísticas”	[“0”, “1”, “2”, “3”, “4”]. Muestra mensaje de error.
mostrar_menu_principal	Funcion	Define la estructura del menú principal.	[“0”, “1”, “2”, “3”, “4”]. Muestra mensaje de error.
main	Funcion	Contiene el código del programa principal.	

Tabla 4: Diccionario de datos (funciones).

Enlace al video explicativo.

https://drive.google.com/file/d/1StA7RnnXzAA3yqP06EnY_-bs-aJD8WBh/view?usp=sharing

Dificultades y conclusiones

Durante el desarrollo del proyecto se aplicaron los conceptos fundamentales de Programación I relacionados con estructuras de datos, funciones, modularización, validaciones y manejo de archivos CSV. Asimismo, fue necesario afrontar distintos desafíos vinculados con la implementación de funcionalidades, la organización del código y la elaboración de la documentación. Estos aspectos fueron resueltos progresivamente mediante la investigación, la práctica y la colaboración entre los integrantes del equipo.

Además, el proyecto permitió incorporar herramientas de apoyo al desarrollo y la organización, como Git, GitHub y Jira Software, facilitando el control de versiones, la gestión de tareas y el seguimiento de las distintas actividades realizadas.

La realización de este Trabajo Práctico Integrador contribuyó al fortalecimiento de habilidades de resolución de problemas, organización del trabajo y desarrollo de programas en Python, permitiendo consolidar los conocimientos adquiridos durante la cursada.

Bibliografía

Información de los países extraída de la base de datos de las Naciones Unidas

<https://data.un.org/es/index.html>

Material de la materia Programación I de la Tecnicatura Universitaria en Programación.

Informacion sobre archivos csv: <https://docs.github.com/>